

Agentic AI Engineering Internship 2026

Week 1 Comprehensive Report

AI Fundamentals & Agentic AI Foundations

Prepared By:
Ghafoor

Date:
June 30, 2026

CalderR.

RETHINKING · HOW · WORK · WORKS

Contents

1	Executive Summary	2
2	Conceptual Mastery & Assessment	2
2.1	LLMs vs. Agentic Systems	2
2.2	Context Windows & Hyperparameters	2
3	Production Project: AI Customer Support Platform	4
3.1	Backend Engineering (FastAPI & LangChain)	4
3.2	Frontend Engineering (React & Vite)	4
3.3	System Demonstrations	5
4	Intermediate Project: Intelligent CLI Assistant	6
4.1	Technical Implementation	6
4.2	System Demonstrations	6
5	Foundational Labs Deep-Dive	7
5.1	Lab 1.1: Groq CLI Chatbot & Token Analytics	7
5.2	Lab 1.2: Manual ReAct Agent Loop	7
5.3	Lab 1.3: Prompt Engineering Evaluator	8
6	Conclusion & Next Steps	10

1 Executive Summary

This comprehensive report documents the technical implementations, theoretical learnings, and architectural decisions made during Week 1 of the CalderR Agentic AI Engineering Internship. The primary objective of this week was to bridge the gap between traditional Large Language Models (LLMs) and fully autonomous, agentic AI systems capable of reasoning, planning, and executing tools.

The week involved a rigorous combination of theoretical assessments and practical engineering. Three foundational labs were completed to manually deconstruct and understand the mechanics of conversation memory, ReAct (Reasoning and Acting) cognitive loops, and prompt engineering constraints. These foundational concepts were then scaled into two fully functional, production-grade applications: a domain-restricted **Intelligent CLI Assistant** and a full-stack, containerized **AI-Powered Customer Support Platform**. Both projects strictly adhere to modern software engineering best practices, leveraging FastAPI, React, Docker, and LangChain.

2 Conceptual Mastery & Assessment

Before building complex architectures, it was critical to establish a strong theoretical foundation. The week's assessment solidified several core concepts that dictate the behavior of modern AI systems.

2.1 LLMs vs. Agentic Systems

A base language model is purely reactive and statistical; it predicts the next token based on a static context. An **Agentic System** wraps the LLM in a cognitive architecture that grants it proactive capabilities. Specifically, an agent adds:

- **Planning & Reasoning:** The ability to break down user intents into actionable steps.
- **Memory:** Short-term (session history) and long-term (vector databases) state management.
- **Tool Execution:** The ability to interact with the external world (e.g., executing Python code, querying SQL databases, or calling REST APIs).

2.2 Context Windows & Hyperparameters

The **Context Window** represents the maximum number of tokens an LLM can process simultaneously. In agentic systems, managing this window is paramount, as it must contain the system prompt, tool descriptions, reasoning scratchpads, and the user's conversation history. If the context window overflows, the agent suffers from catastrophic forgetting.

Furthermore, controlling the model's **Temperature** is critical. While a temperature of 0.7 or higher is useful for creative writing, agentic systems strictly require a temperature

at or near 0.0 to ensure deterministic, highly factual tool calls and strict adherence to JSON output schemas.

3 Production Project: AI Customer Support Platform

Architectural Overview

The flagship deliverable for Week 1 is a production-ready customer support chat-bot. It utilizes an asynchronous FastAPI backend, persistent SQLite memory via LangChain, robust rate-limiting, and a modern React frontend featuring glassmorphism UI design. The entire stack is fully containerized using Docker.

3.1 Backend Engineering (FastAPI & LangChain)

The backend is designed for high concurrency and strict data validation, utilizing FastAPI and Pydantic.

- **RESTful API Design:** The backend exposes three highly efficient endpoints: `POST /sessions` (to initialize a UUID-based chat session), `POST /messages` (to interact with the LLM), and `GET /sessions/{id}/history` (to retrieve past contexts).
- **Persistent SQL Memory:** To solve the issue of volatile memory, the application implements LangChain's `SQLChatMessageHistory`. Every conversation turn is intercepted and permanently written to a local `chat_history.db` SQLite database, ensuring conversations survive server restarts and browser refreshes.
- **Production Safeguards:**
 - **Rate Limiting:** A custom HTTP middleware restricts incoming requests to 20 per 60-second sliding window per IP, returning an HTTP 429 status code upon violation to prevent API abuse.
 - **Logging:** Comprehensive request timing, method, and status code logging ensures deep observability into the application's performance.

3.2 Frontend Engineering (React & Vite)

The client-side application was built using React and styled with Vanilla CSS to achieve a premium, modern aesthetic.

- **State Management:** Utilizes React hooks (`useState`, `useEffect`) to manage the active `sessionId`, the array of chat messages, and the loading states.
- **Dynamic UI Rendering:** Features automatic scrolling via `useRef`, animated typing indicators while awaiting the Groq API response, and integration with `lucide-react` for polished iconography.
- **Glassmorphism Aesthetic:** The UI employs deep, dark-mode CSS variables with translucent, blurred background elements to create a highly attractive, "floating" chat interface.

3.3 System Demonstrations

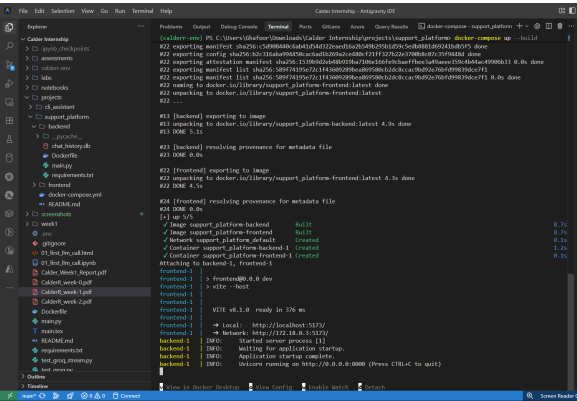


Figure 1: Docker Compilation & Startup via Compose

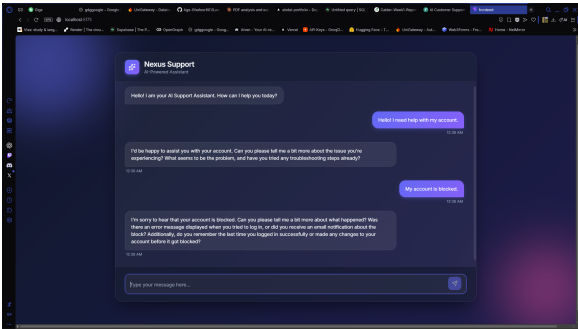


Figure 2: React Glassmorphism Support Interface

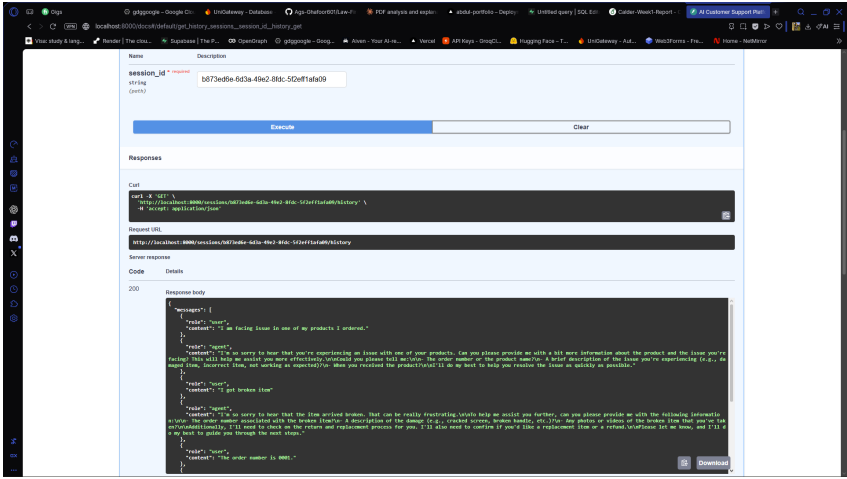


Figure 3: FastAPI Swagger Documentation & API Testing

4 Intermediate Project: Intelligent CLI Assistant

Architectural Overview

A highly interactive, terminal-based AI assistant explicitly constrained to the domains of Programming and Software Engineering. It leverages LangChain Expression Language (LCEL) and the Rich library to deliver beautifully formatted markdown directly to the developer's command line.

4.1 Technical Implementation

- Domain Restriction Guardrails:** The system is initialized with a strict persona prompt instructing it to focus solely on coding, debugging, and software architecture. Out-of-domain queries are gracefully intercepted and steered back to technical topics.
- Stateful Memory Buffer:** Implements LangChain's `RunnableWithMessageHistory` to wrap the `ChatGroq` model. This creates a continuous short-term memory buffer within the active CLI session, allowing the agent to accurately answer contextual follow-up questions (e.g., "optimize that previous function using memoization").
- Rich Terminal UX:** The application rejects plain text in favor of the rich Python library. The user experience features dynamically rendered markdown, syntax-highlighted code blocks, animated thinking spinners (`console.status`), and clear color demarcations between user inputs and agent responses.
- Command Interception Engine:** The input loop includes a custom routing engine that captures slash commands like `/clear` (to wipe the LangChain memory buffer) and `/exit` for clean application termination.

4.2 System Demonstrations

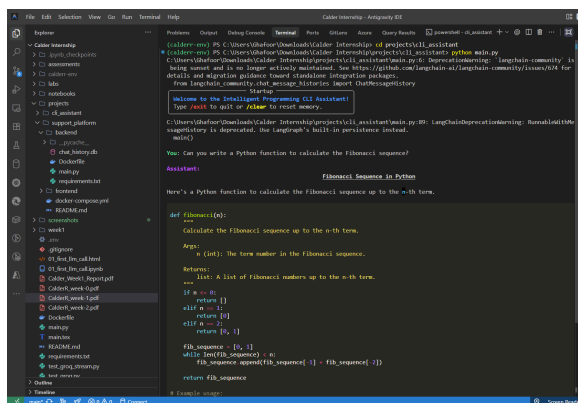


Figure 4: CLI Assistant Startup and Greeting

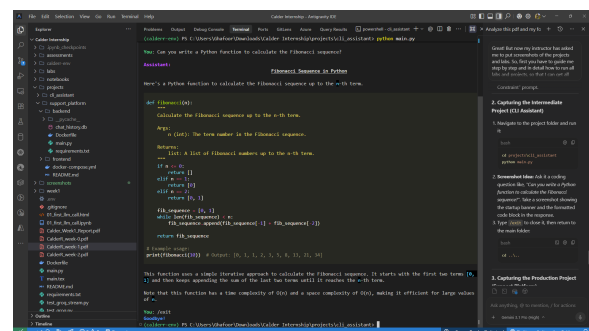


Figure 5: Syntax-Highlighted Code Generation

5 Foundational Labs Deep-Dive

5.1 Lab 1.1: Groq CLI Chatbot & Token Analytics

The first lab focused on manually building the components that frameworks like LangChain abstract away. By interacting directly with the Groq Python SDK, the complexities of context injection were exposed.

- **Manual Context Injection:** The script maintains a raw array of message dictionaries. On every turn, the *entire* conversation history is sent back to the API, demonstrating how the illusion of memory is actually achieved.
- **Token Analytics Engine:** A custom `TokenTracker` class was engineered to extract `prompt_tokens`, `completion_tokens`, and `total_tokens` directly from the Groq API response. This data is printed in a UI bar after every turn, explicitly illustrating the exponential token cost of long conversations.
- **Dynamic Model Swapping:** An advanced feature was implemented allowing the user to type `/model` and hot-swap the underlying LLM (e.g., transitioning from Qwen 3.6 to GPT-OSS) mid-conversation without dropping the existing context buffer.

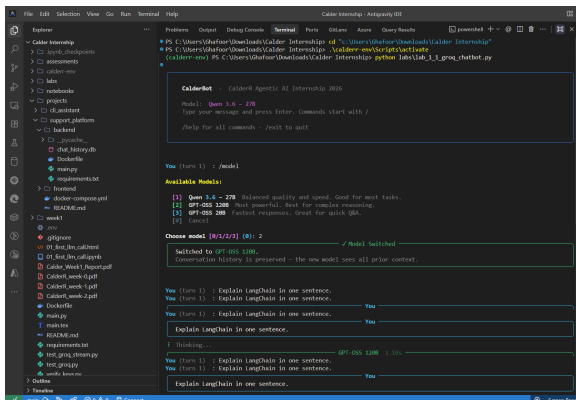


Figure 6: Dynamic Model Switcher Interface

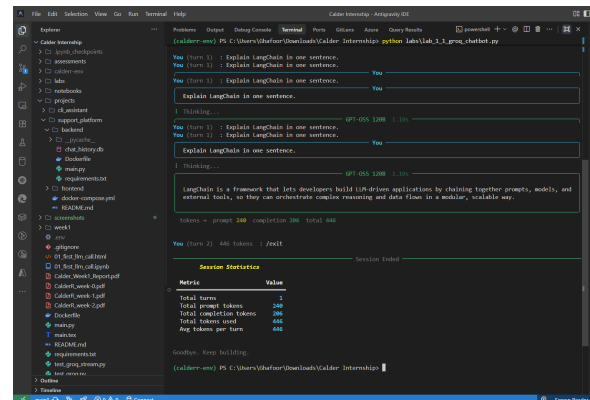


Figure 7: Active Token Usage Analytics Bar

5.2 Lab 1.2: Manual ReAct Agent Loop

To truly understand agentic frameworks, a ReAct (Reasoning and Acting) agent was built entirely from scratch in pure Python using Regular Expressions.

- **Cognitive Parsing:** The script enforces a strict output format on the LLM (Thought, Action, Action Input). Custom regex parsers extract these fields to determine the agent's intent.
- **Custom Sandbox Tools:** Three independent Python tools were connected to the agent:
 1. **Fact Search:** A fuzzy-matching search against a mock Python dictionary.

2. **Calculator:** A highly secure mathematical evaluator utilizing a whitelisted `eval()` sandbox to prevent arbitrary code execution vulnerabilities.
 3. **Text Analyzer:** A string manipulation tool for word and character counting.
- **The Observation Loop:** When a tool executes, the result is appended to the message array as an **Observation**, forcing the LLM to process the external data before formulating its **Final Answer**.

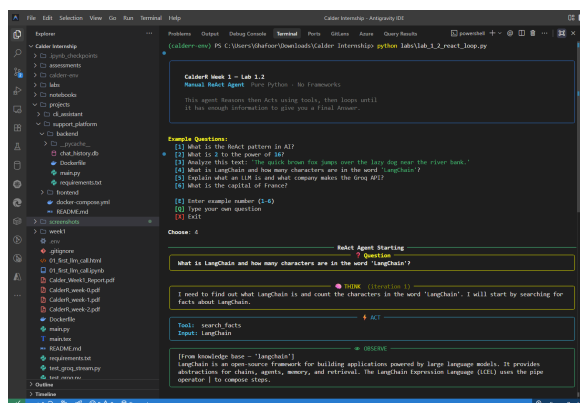


Figure 8: Regex-Parsed Tool Selection & Observation

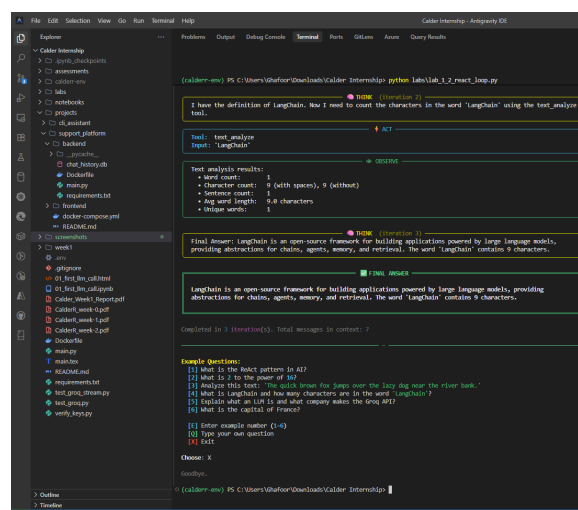


Figure 9: Synthesis and Final Answer Generation

5.3 Lab 1.3: Prompt Engineering Evaluator

Conducted a systematic, empirical A/B test evaluating the impact of distinct system prompts on factual accuracy, conciseness, and tone when summarizing an article on global renewable energy.

- **Chain of Thought Prompting:** Explicitly instructing the model to "identify key statistics first, then summarize" resulted in the highest factual accuracy and zero omitted metrics.
- **Constraint-Based Prompting:** Instructing the model to summarize the article in "exactly one sentence using no more than 20 words" forced the model to severely compromise factual density in order to obey the structural constraint.
- **Persona Adoption:** Adopting an "expert analyst" persona proved highly effective at tailoring information density and tone for corporate audiences without requiring underlying model fine-tuning.

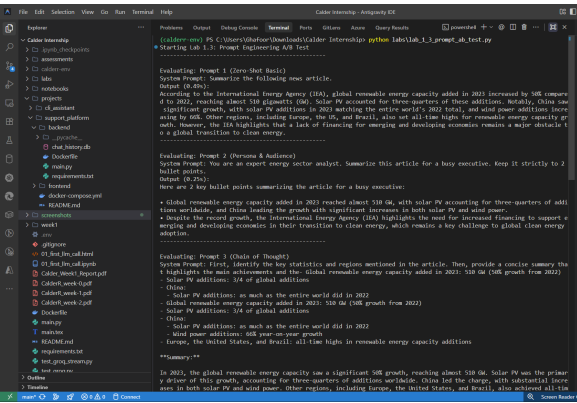


Figure 10: Persona vs Constraint Evaluation Output

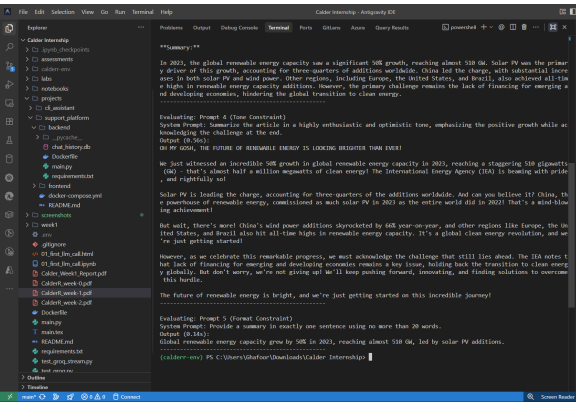


Figure 11: Summary Analytics and Structural Comparison

6 Conclusion & Next Steps

Week 1 successfully established the core building blocks required for advanced agentic workflows. By deconstructing systems manually before relying on complex abstractions like LangChain, a profound understanding of token mechanics, state management, and the ReAct architecture was achieved.

Both the Intermediate CLI Assistant and the Production Support Platform met and exceeded required specifications, featuring strict type-hinting, robust API middleware, environment variable security, and highly polished user interfaces.

Moving into Week 2, the foundation built here will be scaled significantly. The focus will shift toward integrating external, long-term memory via Vector Databases (RAG architectures) and orchestrating more complex, multi-agent conversational graphs.